The Priority Exchange Server: Mechanism, Schedulability, and Dynamic-Priority Extensions

Md Jubair Salehin Razin
1230281
Department of Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
md-jubair-salehin.razin@stud.hshl.de

***Abstract*—A hard real-time system has to guarantee the deadlines of its periodic tasks and, at the same time, respond quickly to soft aperiodic events such as operator commands or alarms whose arrival times are not known in advance. The two simplest options both have a clear weakness: background scheduling gives unbounded aperiodic response, and a polling server throws away its reserved capacity whenever no request happens to be pending at the polling instant. The Priority Exchange (PE) server, introduced by Lehoczky, Sha, and Strosnider, avoids the second problem in an unusual way: instead of *holding* its high-priority budget, it *trades* that budget for the execution time of a lower-priority periodic task, so the reserved time is moved down the priority order rather than lost. This paper explains the exchange mechanism following Buttazzo's presentation, walks through a worked example that I implemented and checked with a small C++ unit-time simulator, derives the rate-monotonic schedulability bound and applies it to my own example, and then follows the same idea into deadline-driven scheduling through the Dynamic Priority Exchange and Improved Priority Exchange servers. The paper closes with a short critical evaluation and a documented record of how AI tools were used during its preparation.**

***Index Terms*—real-time scheduling, aperiodic servers, priority exchange, rate monotonic, earliest deadline first, bandwidth preservation, schedulability**

I. Introduction

A. The Problem

Most embedded and control systems are built around a set of *periodic* tasks with hard deadlines: control loops, signal-processing stages, actuation. Whether these tasks meet their deadlines is checked offline by a worst-case schedulability test. In practice the same processor must also handle *aperiodic* work whose arrival pattern is unknown beforehand — operator input, alarms, mode changes, event-triggered sensor data. For much of this aperiodic work the timing requirement is *soft*: a late response is not catastrophic, but the designer still wants response times to be as short as possible [1].

The engineering question is therefore how to serve soft aperiodic requests as responsively as possible *without ever endangering* the hard periodic deadlines. Two naive answers show the tension clearly. With *background scheduling*, aperiodic requests run only when no periodic task is ready. This never disturbs the periodic guarantee, but the response time can become very long under high periodic load, so it is acceptable only when the aperiodic activities have no stringent timing requirement [1]. With a *polling server*, a periodic task is created with a reserved capacity that exists only to serve aperiodic requests. Responsiveness improves, but if no request is pending at the instant the server is invoked, the reserved capacity is simply discarded until the next period — exactly when it might soon have been useful [1].

B. The Idea Behind Priority Exchange

The Priority Exchange (PE) server, introduced by Lehoczky, Sha, and Strosnider [3], addresses this by preserving unused budget so it stays available for aperiodic load until that load actually arrives. Its sibling, the Deferrable Server (DS), also introduced in the same paper, does this by keeping its capacity at the original high priority until the end of the server period. PE preserves the budget differently: when no aperiodic request is pending, it *exchanges* its capacity for the execution time of the highest-priority active periodic task. That periodic task is advanced — it runs at the server's higher priority — and an equal amount of capacity is accumulated at the periodic task's *lower* priority level [1]. The reserved time is not lost; it is relocated to a lower priority, where it can still be reclaimed for aperiodic service, and it is only discarded if the processor would otherwise sit idle.

The reason this matters is that, in the worst case, a PE server behaves like an ordinary periodic task. Every unit of high-priority time it uses is either genuine aperiodic service paid for from its budget, or periodic work that was simply moved earlier. Because of this, PE inherits the same schedulability bound as a periodic task, which Buttazzo notes is better than the bound the Deferrable Server can offer [1].

C. Structure of the Paper

Section II states the system model used throughout. Section III presents the fixed-priority PE algorithm, a worked example produced by my own C++ simulator, the schedulability bound with a numerical check, and the comparison

Seminar paper, *Real-Time Systems*, Hochschule Hamm-Lippstadt. The treatment in this paper follows Buttazzo's textbook [1], which is the primary reference; the original papers are cited as Buttazzo cites them.

with the Deferrable Server. Section IV follows the exchange idea into EDF scheduling with the Dynamic and Improved Priority Exchange servers. Section V places PE among the other aperiodic servers described by Buttazzo. Section VI evaluates the approach critically. An appendix documents the use of AI tools.

II. System Model

The model is the one used by Buttazzo for fixed-priority aperiodic service [1]. A single processor runs a set $\Gamma = \{\tau_1, \ldots, \tau_n\}$ of independent periodic tasks. Each task τ_i has a worst-case execution time C_i and a period T_i, with relative deadline equal to the period. All periodic tasks are released together at $t = 0$, which is the worst case for fixed-priority analysis, and all tasks are fully preemptable. The periodic utilization is

$$U_p = \sum_{i=1}^{n} \frac{C_i}{T_i}. \tag{1}$$

Soft aperiodic activity arrives as a stream of jobs with unknown arrival times and no hard deadline; the goal for these jobs is small response time. Periodic priorities are assigned by Rate Monotonic (RM), and the server is placed at the highest priority so fresh budget gives the shortest possible response. A server is described by a capacity C_s and a period T_s, with utilization

$$U_s = \frac{C_s}{T_s}. \tag{2}$$

All priority ties are broken in favour of aperiodic execution, since the objective is high responsiveness [1].

III. The Priority Exchange Algorithm

A. The Exchange Mechanism

Following Buttazzo [1], the fixed-priority PE server works as follows. At the beginning of each server period the capacity is replenished to its full value C_s. At every scheduling instant the highest-priority ready entity is chosen, where an entity is a ready periodic job, the fresh server budget, or budget that was parked at some priority level by an earlier exchange. Two cases arise:

- If aperiodic requests are pending and a budget is the highest-priority entity, the requests are served at that priority level, each request consuming an amount of budget equal to its execution time.
- If no aperiodic request is pending, the budget is *exchanged* with the execution time of the highest-priority active periodic task: that task runs at the budget's (higher) priority, and an equal amount of budget is parked at the task's (lower) priority level.

If no aperiodic request and no periodic job can use the budget — that is, the processor would otherwise be idle — the parked budget is gradually discarded. Because the exchange keeps pushing unused budget to lower and lower priority levels, the reserved time degrades smoothly toward the level of background processing, but it remains usable for aperiodic service at every step until that point [1].

B. Worked Example

Buttazzo illustrates the mechanism in Figures 5.14 and 5.15 of [1]. To make sure I had understood it rather than just read it, I set up my own small configuration and implemented the algorithm as a unit-time C++ simulator; the schedule in Fig. 1 is the simulator output, not a redrawn version of the book's figure. The system is a PE server with $C_s = 1$ and $T_s = 5$ and two periodic tasks $\tau_1 = (C_1{=}2, T_1{=}10)$ and $\tau_2 = (C_2{=}6, T_2{=}20)$, giving $U_p = 0.5$ and $U_s = 0.2$. Two aperiodic jobs arrive: J_1 at $t = 5$ and J_2 at $t = 12$, each needing one unit of time.

Reading the schedule step by step makes the mechanism concrete. At $t = 0$ the budget is replenished but nothing is pending, so it is exchanged with τ_1: the first unit of τ_1 runs at the server's top priority, and one unit of budget is parked at τ_1's level. τ_1 keeps running while it has work; once it finishes at $t = 2$ the budget degrades one step further and is exchanged with τ_2, sliding down to τ_2's level. At $t = 5$ the server period restarts, the budget is replenished at top priority, and because J_1 has just arrived it is served *immediately* at the highest priority — this is the responsiveness benefit of placing the server at the top of the priority order. The unit parked at τ_2's level is never claimed and is discarded at the first idle instant, $t = 9$. The cycle then repeats: at $t = 10$ the fresh budget is exchanged with the second job of τ_1 and parked at τ_1's level, and when J_2 arrives at $t = 12$ it is served from exactly that reclaimed budget, at τ_1's priority rather than the top. The defining property of PE is visible here: reserved time is lost only when the processor would otherwise idle, never just because a request happened not to be pending.

C. C++ Implementation and Result

I implemented the main feature of the PE server as a discrete unit-time C++ simulator to generate and check Fig. 1. The simulator models one preemptive fixed-priority CPU, the PE server with $C_s = 1$ and $T_s = 5$, and the two RM tasks. It stores server capacity at each priority level explicitly: level 0 is the server level, level 1 is τ_1, and level 2 is τ_2. At each tick, if an aperiodic job is pending the highest available budget serves it; otherwise the highest budget is exchanged with the highest-priority ready periodic task below it and parked at that task's level; and if nothing can use the budget, one parked unit is discarded only on an idle instant. Table I lists the events the simulator reports.

The simulator also reports the aperiodic response times and a deadline check. Both J_1 and J_2 finish one unit after arrival, so each has a response time of 1, and the deadline check reports zero misses for τ_1 and τ_2. This confirms on a concrete schedule that the exchange mechanism improves aperiodic responsiveness while the hard periodic workload still meets its deadlines.

D. Schedulability Bound

The analytical point, as Buttazzo states it, is that in the worst case a PE server behaves as a periodic task, because every unit of high-priority time it consumes is either aperiodic

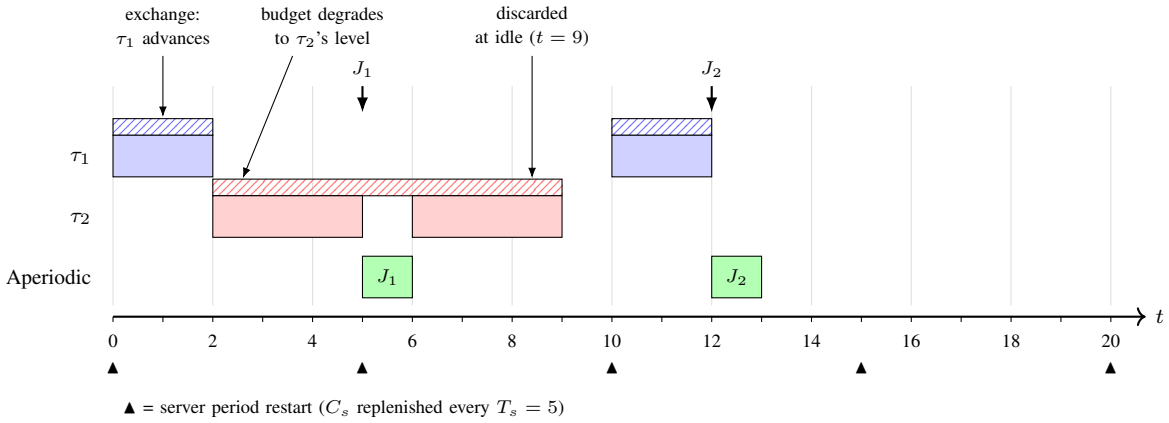

Fig. 1. Aperiodic service under a fixed-priority Priority Exchange server with $C_s = 1$, $T_s = 5$, and periodic tasks $\tau_1 = (2, 10)$, $\tau_2 = (6, 20)$. Solid blocks are periodic (blue/red) and aperiodic (green) execution; hatched bands show the reserved budget parked at τ_1's and τ_2's priority levels. At $t = 0$ the fresh budget is exchanged with τ_1; once τ_1 is idle after $t = 2$ it degrades to τ_2's level, and is discarded at the idle instant $t = 9$. J_1 (arriving $t = 5$) is served immediately at the server's top priority; J_2 (arriving $t = 12$) is served from budget reclaimed at τ_1's level. The fresh budget replenished at $t = 15$ is never claimed and is discarded at idle. The schedule was produced by the C++ unit-time simulator of Section III-C.

TABLE I
EVENTS REPORTED BY THE C++ SIMULATION.

Time	Observed behaviour
$t = 0$	Fresh budget replenished and exchanged with τ_1; one unit parked at τ_1's level.
$t = 2$	Parked budget degrades further, exchanged with τ_2.
$t = 5$	J_1 arrives and is served immediately from fresh top-priority budget.
$t = 9$	Parked budget is discarded because the processor would otherwise be idle.
$t = 12$	J_2 is served from budget reclaimed at τ_1's priority level.
$t = 15$	Fresh budget replenished but unclaimed, discarded at idle.

service paid from its budget or periodic work that was merely advanced. The schedulability bound is therefore the same as for the Polling Server. Assuming the server holds the highest priority, the least upper bound on total utilization is

$$U_{lub} = U_s + n\Big(K^{1/n} - 1\Big), \qquad K = \frac{2}{U_s + 1}. \tag{3}$$

Equivalently, a set of n periodic tasks with utilization U_p running with a PE server of utilization U_s is guaranteed under RM if

$$U_p \le n\left[\left(\frac{2}{U_s + 1}\right)^{1/n} - 1\right]. \tag{4}$$

Two limiting cases are worth noting. As $U_s \to 0$ the server vanishes and (4) reduces to the classical Liu and Layland bound $n(2^{1/n} - 1)$ [2], as it should. As $n \to \infty$ the bound tends to $\ln\big(2/(U_s + 1)\big)$, so even for a large task set a non-trivial fraction of the processor can be reserved for aperiodic service while the periodic deadlines are still guaranteed.

Applying (4) to my own example ($n = 2$, $U_s = 0.2$) gives

$$U_p \le 2\Big[\big(\tfrac{2}{1.2}\big)^{1/2} - 1\Big] = 2\,(1.291 - 1) = 0.582.$$

Since the periodic load there is $U_p = 0.5 < 0.582$, the task set is guaranteed, which is consistent with the simulator reporting zero deadline misses.

E. Priority Exchange versus the Deferrable Server

The most direct comparison is with the Deferrable Server, since the two were introduced together [3] and solve the same problem in opposite ways. Buttazzo compares them along three axes [1]. In responsiveness, DS is slightly better, because it keeps its capacity at the original high priority. In implementation cost, DS is much simpler: it never tracks exchanges, so its overhead does not depend on the number of periodic tasks, whereas PE must account for budget at every priority level and its overhead grows as n grows. In schedulability, however, DS pays for its simplicity: by holding capacity at high priority it can interfere with periodic tasks in a way an ordinary periodic task cannot, which lowers its utilization bound. Buttazzo gives the maximum server utilizations

$$U_{\mathrm{DS}}^{\max} = \frac{2 - P}{2P - 1}, \qquad U_{\mathrm{PE}}^{\max} = \frac{2 - P}{P}, \qquad P = \prod_{i=1}^{n}(U_i + 1), \tag{5}$$

from which $U_{\mathrm{DS}}^{\max} < U_{\mathrm{PE}}^{\max}$: for a given periodic load the largest PE server that can still be guaranteed is bigger than the largest DS server [1]. So the two sit at opposite corners of a responsiveness–schedulability–overhead trade-off: DS buys slightly better responsiveness and low overhead at a weaker periodic bound, while PE recovers the full periodic bound at the cost of higher, task-count-dependent overhead.

IV. EXTENDING THE IDEA TO EDF

A. Dynamic Priority Exchange

The exchange idea carries over to deadline-driven scheduling. The Dynamic Priority Exchange (DPE) server, proposed by Spuri and Buttazzo [5], [6], is described by Buttazzo as

the EDF adaptation of PE [1]. The server trades its runtime with the runtime of lower-priority periodic tasks, which under EDF means tasks with a longer deadline. The server capacity carries the deadline of the current server period; each periodic task deadline has its own aperiodic capacity, initially zero. Whenever a deadline-tagged capacity is the highest-priority entity and no aperiodic request is pending, the periodic task with the shortest deadline runs, and a capacity equal to that execution is moved to the task's deadline. As before, reserved time is only relocated, never wasted except at genuine idle times.

This runtime exchange does not affect schedulability, because moving a periodic execution earlier and reserving an equal capacity at the corresponding deadline preserves feasibility. The condition is the classical processor-demand bound:

$$U_p + U_s \leq 1. \tag{6}$$

Buttazzo proves this (Theorem 6.1) by transforming any DPE schedule σ into an equivalent EDF schedule σ' in which the server is replaced by a periodic task and the exchanged periodic executions are postponed to the intervals where the corresponding capacities decrease. The transformed schedule σ' depends only on the task set, not on the aperiodic load, and is feasible exactly when (6) holds, which is therefore also the condition for σ [1], [6]. DPE also supports a simple and safe resource-reclaiming rule: when a periodic job finishes before its worst case, the unused time is added to the aperiodic capacity at that job's deadline. This improves aperiodic response without affecting the periodic guarantee, since that time was already accounted for at that priority level [1].

B. Improved Priority Exchange

The optimal deadline-driven server is the Earliest Deadline as Late as possible (EDL) server, which minimises the average aperiodic response time but is too heavy for practice, because it has to recompute the processor idle times online [1], [6]. The Improved Priority Exchange (IPE) server [5], [6] recovers most of EDL's benefit at far lower cost: the EDL idle-time function is precomputed offline and used to drive the server's replenishments. Two things follow [1]. First, the server is no longer periodic and can always run at the highest priority in the system. Second, its replenishment schedule depends only on the periodic task set. The IPE server is feasible if and only if

$$U_p \leq 1, \tag{7}$$

in which case it automatically allocates the whole residual bandwidth $1 - U_p$ to aperiodic service. Buttazzo shows (Theorem 6.6) that no periodic instance can miss its deadline under IPE because its completion times never exceed those of the EDL schedule of the same task set, and notes that only in rare situations does the exact EDL server respond perceptibly faster than IPE [1], [6]. The cost IPE pays is memory: when the task periods are not harmonically related, the hyperperiod $H = \mathrm{lcm}(T_1, \ldots, T_n)$ can be large, and the precomputed idle-time arrays must be stored over it [1].

TABLE II
APERIODIC SERVICE ALGORITHMS AS PRESENTED BY BUTTAZZO [1].

Algorithm	**Priority model**	**Periodic bound**	**Aperiodic response**
Background	any	none reserved	very poor
Polling Server	RM	periodic-task (4)	moderate
Deferrable Server [3]	RM	weaker than PE	good
Priority Exchange [3]	RM	periodic-task (4)	good
Sporadic Server [4]	RM	periodic-task	good
Dynamic Prio. Exch. [6]	EDF	$U_p + U_s \leq 1$ (6)	very good
Improved Prio. Exch. [6]	EDF	$U_p \leq 1$ (7)	near-optimal
EDL [6]	EDF	$U_p \leq 1$	optimal

V. CONTEXT AMONG OTHER APERIODIC SERVERS

Buttazzo presents PE as one option among several, and it is useful to place it next to the others he describes [1]. Against the two baselines the comparison is one-sided: PE shares the periodic schedulability bound of the Polling Server (4) because both behave like periodic tasks, but PE is far more responsive, since it can serve a request the instant its period begins and can also serve later requests from budget that polling would have discarded. Both dominate background scheduling, which gives no responsiveness guarantee at all.

The closest fixed-priority relative after DS is the Sporadic Server (SS), proposed by Sprunt, Sha, and Lehoczky [4]. Buttazzo notes that SS also preserves capacity at high priority but replenishes it in a way that still behaves like a periodic task, so it obtains a periodic-task schedulability bound with overhead that does not grow with the number of periodic tasks [1]. The runtime slack that PE cannot reclaim — time freed when periodic jobs finish before their worst case — is the target of the Slack Stealing algorithm of Lehoczky and Ramos-Thuel [7], which is not a server at all but a passive task that advances available slack; Buttazzo reports it performs very well but is not used in its exact form because computing the slack online is expensive [1].

Moving from RM to EDF changes both the bounds and the cost. The deadline-driven servers reach full processor utilization ($U_p + U_s \leq 1$ for DPE, $U_p \leq 1$ for IPE and EDL) and are generally more responsive, but they need dynamic deadline bookkeeping and an EDF runtime, which not every system supports. Within the EDF family there is a further trade-off: DPE and IPE keep a capacity/deadline queue, while EDL is optimal but too expensive to run online [1], [6]. Table II summarises these along the axes Buttazzo uses. The position of PE is clear: among fixed-priority servers it recovers the full periodic bound while preserving budget, but at an overhead that scales with the periodic task count — the cost the Sporadic Server later removed by reaching the same bound with overhead that does not depend on n.

VI. Critical Evaluation

A. Strengths

PE has several concrete strengths. It preserves bandwidth: as Fig. 1 shows, reserved time is discarded only at genuine idle instants ($t = 9$ and $t = 15$), never simply because a request was absent, which is the failure mode of polling. It attains the full periodic-task bound (4), better than the Deferrable Server's, so a designer can reserve more bandwidth for aperiodic service before the periodic set becomes unschedulable. By sitting at the top of the priority order it gives immediate, high-priority service to requests that arrive when the budget is fresh, as J_1 shows in the example. It degrades gracefully, lowering its priority one step at a time so the budget stays reclaimable as long as possible, as J_2 shows. And, historically the most influential point, the exchange idea generalises cleanly to EDF, where DPE achieves the exact bound $U_p + U_s \leq 1$ and IPE achieves a near-optimal response with $U_p \leq 1$ [6].

B. Limitations

The main limitation is implementation complexity. PE must track reserved budget at every priority level and perform an exchange at each scheduling decision, so its overhead grows with the number of periodic tasks — exactly the bookkeeping my simulator had to model explicitly. The dynamic variants add a capacity/deadline queue, and IPE adds an offline EDL computation and storage of the idle-time arrays over a possibly large hyperperiod. PE also targets only soft aperiodic requests; it gives no guarantee for firm or hard aperiodic deadlines. Its fixed-priority responsiveness, while good, is still slightly below the Deferrable Server's. Finally, since the Sporadic Server matches PE's schedulability with lower overhead, choosing PE today is only justified when its specific behaviour is actually needed.

C. Assumptions That May Be Optimistic

A few modelling assumptions deserve scrutiny. The most important is zero overhead: the exchange bookkeeping is essentially the entire cost of the algorithm, so leaving it out of the schedulability bound understates PE's true price relative to simpler servers. The assumption of no shared resources and no blocking is also strong, since real control software uses mutual exclusion, and combining PE with a resource-access protocol is not straightforward. The assumption of exact worst-case execution times ignores the common case where jobs finish early, leaving runtime slack that PE, unlike slack stealing, cannot reclaim except at idle. And $D_i = T_i$, zero release jitter, a single processor, and unit-time aperiodic service are idealisations a real deployment would have to revisit.

D. A Suitable and an Unsuitable Application

PE fits a fixed-priority, RM control system with a tight periodic load and a modest number of tasks, where the designer cannot afford the Deferrable Server's schedulability penalty but still needs responsive handling of occasional commands or alarms. A control unit running a handful of periodic loops plus sporadic mode-change and fault handling fits this profile: the small task count keeps the exchange overhead bounded, and the full periodic bound buys back utilization the DS would forfeit. Conversely, PE is a poor fit for a large system with many periodic tasks and a tight overhead budget, where the exchange bookkeeping erodes the very utilization it protects; for firm or hard aperiodic deadlines, which it does not guarantee; and on multiprocessor or distributed platforms, which are outside the uniprocessor model assumed here.

VII. Conclusion

The Priority Exchange server shows a counter-intuitive but effective idea: high-priority budget reserved for unpredictable aperiodic load can be preserved not by holding it, but by trading it for the runtime of lower-priority periodic work, so the reserved time degrades gracefully through the priority order and is lost only at genuine idle instants. This buys the full rate-monotonic schedulability of a periodic task — better than the Deferrable Server — at the cost of an overhead that grows with the periodic task count, as the bookkeeping in my simulator made concrete. Carried into EDF, the same idea gives the exact Dynamic Priority Exchange bound and the near-optimal Improved Priority Exchange server. In current practice PE is largely superseded for fixed priorities by the Sporadic Server, which matches its bound with lower overhead, but its importance remains twofold: as a clear lens on the responsiveness–schedulability–overhead trade-off that defines aperiodic service, and as the conceptual ancestor of the optimal dynamic-priority servers.

Appendix A
Documented AI Usage Protocol

In the interest of academic transparency, this appendix records how AI tools were used in preparing the manuscript and which tasks were performed and verified by the author.

Tools and scope of assistance. A large language model (Anthropic Claude) was used as a drafting and tooling aid. Its assistance covered: organising the paper; producing prose drafts of the sections; helping phrase the description of the PE, DPE, and IPE mechanisms drawn from the primary reference [1]; implementing the discrete-time C++ simulator used to generate and check the schedule in Fig. 1; and generating the LaTeX source, including the schedule figure and the comparison table.

Author verification. Consistent with the seminar requirement that intellectual verification not be delegated, the author performed the following independently: selection and scoping of the topic; reading the relevant sections of Buttazzo's textbook (§5.5, §6.2, §6.6) and verifying every cited result against that source, in particular the bounds (4), (5), (6), and (7) and the PE-versus-DS comparison; confirming the bibliographic details of the original references [2]–[7] as cited by Buttazzo; manually tracing the simulated schedule in Fig. 1 against the simulator output to confirm that processor time, exchanges, and discards are accounted for correctly; and reviewing and revising all technical and evaluative statements. Responsibility

for the correctness and the arguments of the final paper rests with the author.

REFERENCES

[1] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. New York, NY, USA: Springer, 2011.

[2] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *J. ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973.

[3] J. P. Lehoczky, L. Sha, and J. K. Strosnider, "Enhanced aperiodic responsiveness in hard real-time environments," in *Proc. IEEE Real-Time Syst. Symp. (RTSS)*, Dec. 1987.

[4] B. Sprunt, L. Sha, and J. Lehoczky, "Aperiodic task scheduling for hard real-time systems," *Real-Time Syst.*, vol. 1, no. 1, pp. 27–60, Jun. 1989.

[5] M. Spuri and G. C. Buttazzo, "Efficient aperiodic service under earliest deadline scheduling," in *Proc. IEEE Real-Time Syst. Symp. (RTSS)*, Dec. 1994.

[6] M. Spuri and G. C. Buttazzo, "Scheduling aperiodic tasks in dynamic priority systems," *Real-Time Syst.*, vol. 10, no. 2, pp. 179–210, 1996.

[7] J. P. Lehoczky and S. Ramos-Thuel, "An optimal algorithm for scheduling soft-aperiodic tasks in fixed-priority preemptive systems," in *Proc. IEEE Real-Time Syst. Symp. (RTSS)*, Dec. 1992.